point we'll also have renamed the model `OrderHistory` to `OrderNote`.

Now we decide to drop our development database and reapply all migrations. When we do so, the migrations throw an exception in migration 4: our application no longer contains a class called `OrderHistory`, so the migration fails.

One solution, proposed by Tim Lucas, is to create local dummy versions of the model classes needed by a migration within the migration. For example, the following version of the fourth migration will work even if the application no longer has an `OrderHistory` class:

```
  class CreateOrderHistories < ActiveRecord::Migration

»   class Order < ApplicationRecord::Base; end
»   class OrderHistory < ApplicationRecord::Base; end

    def change
      create_table :order_histories do |t|
        t.integer :order_id, null: false
        t.text :notes

        t.timestamps
      end

      order = Order.find :first
      OrderHistory.create(order: order_id, notes: "test")
    end
  end
```

This works as long as our model classes don't contain any additional functionality that would have been used in the migration—all we're creating here is a bare-bones version.

Defining Indices

Migrations can (and probably should) define indices for tables. For example, we might notice that once our application has a large number of orders in the database, searching based on the customer's name takes longer than we'd like. It's time to add an index using the appropriately named `add_index` method: